

tw-finscope

台灣上市公司財務健康儀表板

Demo: https://FILL_IN_AFTER_DEPLOY.vercel.app **GitHub:** <https://github.com/yukinoshita045/tw-finscope>

1. 做這個的原因

想查一家台灣上市公司的財務狀況，你大概會打開公開資訊觀測站（MOPS）——然後發現自己需要分別下載資產負債表、損益表、現金流量表三份 Excel，每份的日期都是民國年、欄位是中文科目名稱、負數用括號表示、千分位用全形逗號……光是把這三份整理成能比較的格式，就可能花掉一個下午。

tw-finscope 想解決的正是這件事。它把抓資料、清洗、計算指標的過程全部自動化，最後用一個互動儀表板呈現出來，讓你直接用瀏覽器比較不同公司和產業，不需要下載任何東西、不需要懂 Excel。

2. 資料處理流程（ETL）

整個 pipeline 分三個階段，也是這個專案最核心的部分：

抽取（Extract）：用 Python httpx 同時打 MOPS 的三個 API（t164sb03/04/05，分別對應資產負債/損益/現金流量表）。MOPS 有簡單的防爬措施，需要帶上 browser-like headers 才能正常取回資料；遇到網路不穩或被短暫擋掉時，會自動等幾秒後重試，最多三次。

轉換（Transform）：原始資料拿回來之後，有相當多事情要做——把民國年轉成西元年、把中文科目名稱對應到標準化的 key（光這個對應表就寫了 60 幾條）、清掉各種奇怪的格式，最後再計算十項衍生指標：毛利率、營業利益率、淨利率、ROE、ROA、流動比率、負債比率、營業 FCF、以及營收和淨利的同期年增率（YoY）。其中有三個指標會自動判斷是否需要乘以 100 做標準化，因為不同公司的財報有時候單位不一致。

載入（Load）：寫入資料庫時用 ON CONFLICT DO UPDATE，確保不管重跑幾次都不會出現重複資料——這對定期自動排程來說很重要。Schema 設計成三層：最底層是原始長表 statements，中間是整理好的寬表 metrics，最上層是每次 ETL 執行記錄 etl_runs，方便之後追蹤資料來源與更新歷史。

資料規模大約是 40 家公司 × 16 季（2022–2025）× 3 份報表，最終產出約 2,500 筆指標資料。

3. 儀表板與視覺化

儀表板設計的出發點是：讓沒有財務背景的人也能一眼看懂。除了圖表，還設計了一個「財務健康分數」，把五項比率壓縮成 0–100 的單一數字，並依區間顯示顏色（綠/藍/橘/紅），不需要自己去判斷各項數字代表什麼意思。

圖表類型	想讓使用者看到什麼
損益趨勢線圖	多家公司的營收、毛利、淨利放在同一張圖比較，還可以疊加同產業平均值（虛線），一眼看出誰高於行業水準
利潤率長條圖	同時看毛利率、營業利益率、淨利率三層，理解「這家公司賺的錢有多少是真的留下來的」
財務比率雷達圖	五個維度（毛利率/營業利益率/淨利率/ROE/ROA）的多邊形，跨公司比體質最直觀
產業排名表	可切換 9 種指標、按產業過濾，顯示前 25 名，方便快速找出某個產業裡的強者

財務健康分數卡（自訂複合指標）

加權計算：毛利率（30%）+ ROE（25%）+ 流動比率（20%）+ 低負債（15%）+ 營收成長（10%），滿分 100 分，用顏色即時反映健康程度。

4. 資料自動更新

儀表板的資料不是靜態的。專案裡設定了 GitHub Actions 排程，每週一早上 10 點（台北時間）會自動重新執行整個 ETL 流程、更新資料庫，你不需要做任何事。如果想手動觸發（例如剛上線、或者

想補抓特定公司)，也可以直接在 GitHub Actions 頁面按一下「Run workflow」，選好公司代號和年份就會跑起來。

每次 ETL 執行完畢，結果都會寫進 `etl_runs` 表，記錄開始時間、結束時間、處理了幾家公司、寫入了幾筆資料、成功還是失敗。儀表板右上角的「資料更新」小標籤就是直接讀這張表，讓使用者隨時知道現在看到的資料是幾時更新的。

5. 架構與技術選擇

GitHub Actions (每週自動 / 手動觸發) → Python ETL → PostgreSQL on Render → FastAPI on Render → React + Chart.js on Vercel

前後端分開部署的好處是各自可以獨立擴充：後端 API 加新指標不影響前端，前端改 UI 也不需要動到 ETL 邏輯。本地開發時預設用 SQLite，不需要架 Postgres 也能立刻跑起來。

後端 Python 3.12 · FastAPI · SQLAlchemy 2 · httpx · tenacity

前端 React 18 · Vite · Chart.js + react-chartjs-2

資料庫 PostgreSQL 16 (雲端) / SQLite (本地開發)

部署 GitHub Actions · Render (API + DB) · Vercel (前端)

6. 連結

Live Demo https://FILL_IN_AFTER_DEPLOY.vercel.app

API 文件 (Swagger) <https://tw-finscope-api.onrender.com/docs>

GitHub 原始碼 <https://github.com/yukinoshita045/tw-finscope>

ETL 排程記錄 <https://github.com/yukinoshita045/tw-finscope/actions>